HUM1012 : Logic and Language Structure

(LT)

Slot : B21 (Mon 04.25 pm – 05.55 pm) + E14 (Tue 02.50 pm – 04.20 pm)
D21 (Tue 01.15 pm – 02.45 pm) B14 + (Wed 02.50 pm – 04.20 pm)

Venue : AB02 – 301 (B21+E14)
AB02 – 301 (B14+D21)

Prepared and Presented by

Dr. Velmani Ramasamy, M.E., Ph.D., SMIEEE., MACM.,

Associate Professor (Grade 2)

School of SCAI,

VIT Bhopal University,

Bhopal, Madhya Pradesh.

MODULE 3 Set theory, Ordered pairs and Relations, Function 6

Membership, subset, superset, intersection, union, binary relation, ternary relation, quaternary relation, domain, co-domain, symmetry, transitivity, reflexive, characteristic function, function and relation differentiation in Natural Language.

COURSE OUTCOMES:

CO3: Understanding the basics of mathematical logic and its application to natural language semantics.

3.1 Set Theory

- A set is a collection of well-defined and distinct objects, called elements or members.
- A set usually write with curly brackets {}.

  Example:

  A={1,2,3} B = {2,3,1}

  Here, 1,2,3 is an element of A.

  2,3,1 is an element of B.

(a) Membership (∈)

- Denotes an element belongs to a set.
- Notation: x ∈ A means "x is a member of set A".

  Example:

  2∈A ✔, 5 ∉ A ? .

(b) Subset (⊆)

- $A \subseteq B$ means **all** elements of A are also in B.

Example:

$A = \{1,2,3\}, \; B = \{2,3,1\} \rightarrow A \subseteq B.$

(c) Superset (⊇)

- $A \supseteq B$ means A contains **all** elements of B.

Example:

$A = \{1,2,3\}, \; B = \{2,3\} \rightarrow A \supseteq B.$

(d) Intersection (∩)

- Common elements between two sets (no repetition).

Example:

$A = \{1,2,3\}, \quad B = \{2,3,1\} \qquad A \cap B = \{1,2,3\}.$

(e) Union (∪)

- All elements from both sets (no repetition).

Example:

$A = \{1,2,3\}, \quad B = \{2,3,1\} \qquad A \cup B = \{1,2,3\}.$

Set Theory

Concept	Symbol	Meaning	Example
Membership	$\in$	element in set	$3 \in \{1,2,3\}$
Subset	$\subseteq$	all elements of A in B	$\{1,2\} \subseteq \{1,2,3\}$
Superset	$\supseteq$	A contains B	$\{1,2,3\} \supseteq \{1,2\}$
Intersection	$\cap$	common elements	$\{1,2\} \cap \{2,3\} = \{2\}$
Union	$\cup$	all elements combined	$\{1,2\} \cup \{2,3\} = \{1,2,3\}$

3.2 Binary Relation

- A binary relation on a set is a collection of ordered pairs of elements from that set.
- A binary relation connects pairs of elements.
- Formally, if we have two sets A and B, a binary relation R from A to B is a subset of the Cartesian product A×B

$$R \subseteq A \times B$$

If (a, b) ∈ R, we say "a is related to b" under relation R.

Examples:

(a) Set A = {1,2,3}

Define relation R on A as:

$$R=\{(1,2), (2,3), (3,1)\}$$

- Here, 1 is related to 2, 2 is related to 3, and 1 is related to itself.

(b) On real numbers

- Relation $R = \{ (x, y) \in R \times R \mid x < y \}$

  → means "less than" relation.

(c) On students set

- Let, S = {Alice, Bob, Carol}.
- Define relation "is friend of":

  R={(Alice, Bob),(Bob, Carol), (Carol,Alice)}

Types of Binary Relations

- On a set ‘A’, a binary relation ‘R’ can have special properties:

(a) Reflexive

- Every element is related to itself.

  $(a, a) \in R$ for all A.

  Example: “≤” on real numbers..

(b) Irreflexive

- No element is related to itself.

  Example: “<” on real numbers.

(c) Symmetric

- If (a,b), then (b,a) ∈ R.

  Example: “is a sibling of”.

(d) Asymmetric

- If (a,b) ∈ R, then (b,a) ∉ R.

  Example: "is the parent of".

(e) Antisymmetric

- If (a,b) ∈ R and (b,a) ∈ R a=b.

  Example: "≤" on numbers.

(e) Transitive

- If (a,b) ∈ R and (b,c) ∈ R, then (a,c) ∈ R.

  Example: "≤" relation again.

Special Classes of Relations

(a) Equivalence Relation

Equivalence Relation = Reflexive + Symmetric + Transitive

Example: “is congruent modulo n”.

(b) Partial Order Relation

Partial Order Relation = Reflexive + Antisymmetric + Transitive

Example: “≤” on integers.

(c) Total Order

Total Order = Partial order + Comparability (every two elements comparable).

Example: natural numbers with “≤”.

Applications of Binary Relations

Mathematics & Logic

Define ordering (≤, ≥)

Define equivalence classes (modulo arithmetic)

Computer Science

Databases: relations between entities (e.g., “student enrolled in course”).

Graphs: adjacency relation between nodes is a binary relation.

Automata theory: state transitions.

AI & Knowledge Representation

Representing relationships in knowledge graphs (e.g., “Paris is Capital Of France”).

Real-life

Social networks (“is friend of”).

Family tree (“is parent of”).

E-commerce (“customer purchased product”).

3.3 Ternary Relation

- A ternary relation is a relation between three sets (or three elements).
- Formally, if A, B, C are sets, then a ternary relation R is a subset of their Cartesian product:

$$R \subseteq A \times B \times C$$

- i.e., R is a collection of ordered triples (a, b, c), where $a \in A$, $b \in B$, $c \in C$.
- If $(a, b, c) \in R$, we say "a, b, and c are related under R."

Examples:

(a) Database Example (Student–Course–Grade)

Sets:

S = {Alice, Bob} (Students) C = {Math, Physics} (Courses) G = {A,B,C} (Grades)

Relation:

R = {(Alice, Math, A), (Bob, Physics, B)}

→ means Alice got grade A in Math, Bob got grade B in Physics.

(b) Family Relationship Example (Parent–Child–Relation Type)

- Relation $R = \{ (x, y) \in R \times R \mid x < y \}$

  → means "less than" relation.

  (John, Mary, Father) → John is the father of Mary.

  (Anna, Mary, Mother) → Anna is the mother of Mary.

(c) Geometry Example (Point–Line–Incidence)

- Relation between a point, a line, and whether the point lies on that line.

  Example: (P1,L1,True).

Types of Ternary Relations

- Unlike binary relations (where we have reflexive, symmetric, etc.), ternary relations are less about algebraic properties.
- More about role constraints between three entities.

Some important types:

One-to-One-to-One (1-1-1)

- Each triple is unique with no repetition.

  Example: Each student gets exactly one grade in one course.

One-to-Many-to-Many (1-M-M)

- One entity relates to multiple combinations of the other two.

  Example: A teacher may teach many students in many courses.

Many-to-Many-to-Many (M-M-M)

- Any element can combine freely with others.

  Example: Social networks (Person A tags Person B in Picture P).

Applications of Ternary Relations

Databases

Represent relationships among three entities.

Example: Enrolled(Student, Course, Semester).

Computer Science

Graphs with edge attributes (e.g., road between two cities with travel time).

Represented as (City1, City2, Distance) (City3, City4, Distance).

Access Control in security: (User, File, Permission).

Mathematics & Logic

Ternary operations (like the conditional operator in programming:

(condition, value_if_true, value_if_false).

Real-Life

E-commerce: (Customer, Product, Payment Method).

Sports: (Player, Match, Score).

GPS: (Location, Time, Speed).

3.4 Quaternary Relation

- A quaternary relation is a relation among four sets (or four elements).
- Formally, if A, B, C, D are sets, then a quaternary relation R is a subset of their Cartesian product:

$$R \subseteq A \times B \times C \times D$$

That means R is a set of ordered 4-tuples:

(a, b, c, d)

with $a \in A$, $b \in B$, $c \in C$, $d \in D$.

Examples

E-commerce Example (Customer–Product – PaymentMethod – DeliveryStatus)

(John, Laptop, CreditCard, Delivered)

(Mary, iPhone, UPI, Pending)

Examples

Database Example (Student–Course–Semester–Grade)

Sets:

S = {Alice, Bob} (Students)

C = {Math, Physics} (Courses)

T = {Sem1,Sem2} (Semesters)

G={A,B,C} (Grades)

Relation:

R = {(Alice, Math, Sem1, A), (Bob, Physics, Sem2, B)}

→ means Alice got grade A in Math in Semester 1.

Logistics Example (Supplier–Product–Warehouse–Quantity)

(Supplier1, Steel, Warehouse3, 100) → Supplier1 delivered 100 units of Steel to Warehouse3.

Social Network Example (User1–User2–Post–Reaction)

(Alice, Bob, PostX, Like) → Alice liked Bob's PostX.

Types of Quaternary Relations

- Like ternary relations, quaternary relations are not usually defined by symmetry/reflexivity (since those belong to binary).
- Instead, they are categorized based on role constraints:

1–1–1–1 (One-to-One-to-One-to-One)

- Each tuple is unique across all four sets.

  Example: A unique exam result for each student, in each course, semester, and grade.

1–M–M–M (One-to-Many-to-Many-to-Many)

- One entity links to multiple combinations of the other three entities.

  Example: One customer can order multiple products with various payment methods and delivery statuses.

M–M–M–M (Many-to-Many-to-Many-to-Many)

- Any element can combine freely with the other three element.

  Example: In a social network, multiple users can interact with multiple users on many posts in many ways (likes, comments, shares).

Applications of Quaternary Relations

Databases

Modeling relationships among four entities.

Example: Exam(Student, Course, Semester, Grade).

Computer Science

Access control models: (User, File, Operation, Permission).

Transaction logs: (User, Action, Time, Device).

Artificial Intelligence / Knowledge Representation

Knowledge graphs with four-way relationships.

Example: (Doctor, Patient, Disease, Treatment).

Business & Logistics

Supply chain management: (Supplier, Product, Warehouse, Quantity).

E-commerce order tracking: (Customer, Product, Payment, Status).

Real Life

Sports stats: (Player, Match, Team, Score).

Transport: (Flight, Source, Destination, Time).

3.5 Domain

- Domain = The type/area/field of language data the NLP system is working on.
- In NLP, domain refers to the source or nature of text corpus (what kind of text is being processed).
- NLP models are often domain-sensitive — performance depends on the training corpus.

Example:

Tweets → "Social media domain"

Court judgments → "Legal domain"

Medical reports → "Healthcare domain"

Exam Results → "Education Domain"

School, College, University → "Education Domain"

Website → "Internet Domain"

Financial transactions, Banking, → "Finance Domain"

Types of Domains

Domains can be categorized in multiple ways:

(A) By Coverage

General-Domain NLP

Trained on open-domain corpora (Wikipedia, news, books).

Example: General-purpose translation (Google Translate).

Domain-Specific NLP

Focused on specialized areas with unique vocabulary.

Examples:

- ✓ Legal NLP (case law summarization, contract analysis)
- ✓ Medical NLP (disease extraction from clinical reports)
- ✓ Financial NLP (stock prediction, fraud detection)

(B) By Task

Text Classification Domain

Emails, reviews, social media posts → Spam filtering, sentiment analysis.

Information Extraction Domain

News articles, biomedical texts → Named Entity Recognition (NER), Relation Extraction.

Dialogue & Conversational Domain

Chatbots, voice assistants → Task-oriented Vs open-domain dialogue.

Multimodal Domain

Text + Image/Video → Captioning, Visual QA.

(C) By Data Source

Social Media Domain → Tweets, Facebook posts, Reddit.

Scientific Domain → Research papers , Research papers, patents, copyrights.

Business Domain → Customer reviews, Market Analysis, e-commerce queries.

Healthcare Domain → Electronic Health Records (EHRs), clinical notes.

Education Domain → Student essays, online courses.

Tools for Domain NLP

- Tools vary depending on whether we work with general or domain-specific NLP.

General-purpose NLP Tools

NLTK (tokenization, PoS tagging, Tree parsing)

spaCy (NER, dependency parsing, embeddings)

Stanford CoreNLP (syntax, sentiment, coreference)

Hugging Face Transformers (BERT, GPT, T5 for various tasks)

Domain-specific Tools

SciSpacy → Biomedical NLP (disease, drug recognition).

MetaMap (from NIH) → Maps text to UMLS medical concepts.

Legal BERT / CaseLaw BERT → Legal NLP.

Fin BERT → Financial sentiment analysis.

Bio BERT / Clinical BERT → Medical domain.

Knowledge Base & Ontology Tools

WordNet (lexical database for English)

UMLS (Unified Medical Language System for healthcare)

Wikidata / DBpedia (general knowledge graphs)

Protégé + OWL/RDF tools (for domain ontologies)

Applications of Domains in NLP

General-Domain Applications

- ✓ Machine Translation (Google Translate)
- ✓ Sentiment Analysis (Twitter trends, reviews)
- ✓ Chatbots & Assistants (Siri, Alexa)

Domain-Specific Applications

- ✓ Healthcare → Disease diagnosis from patient notes, drug-interaction extraction
- ✓ Legal → Summarizing judgments, contract clause extraction
- ✓ Finance → Fraud detection, financial sentiment analysis, stock prediction
- ✓ Education → Automated essay scoring, plagiarism detection

Cross-Domain Applications

- ✓ Domain Adaptation → Fine-tuning a general model (like BERT) on specialized data.
- ✓ Example: Adapting GPT or BERT from Wikipedia corpus to clinical notes for medical NLP.

3.6 Co-Domain

- In mathematics, the co-domain of a function is the set of possible outputs.
- In NLP,

  Co-domain = the type of output a model/system produces from text.

Example:

Sentiment Analysis:

Domain = Text reviews

Co-domain = {Positive, Negative, Neutral} (labels)

Machine Translation:

Domain = English sentences

Co-domain = French sentences

NER (Named Entity Recognition):

Domain = Raw text

Co-domain = Entities {Person, Location, Organization}

Types of Co-domain in NLP

- Depending on the **task**, the co-domain can take different forms:

(A) Categorical Output

- Finite set of classes/labels.

Examples:

Spam Detection → {Spam, Not Spam}

Sentiment → {Positive, Negative, Neutral}

POS tagging → {Noun, Verb, Adjective…}

(B) Structured Output

- More complex structures, not just labels.

Examples:

Parse Trees (Syntax trees from parsing)

Dependency Graphs (relations between words)

NER outputs → e.g., ("Paris", LOCATION)

(C) Sequence Output

- Output is a sequence of tokens.

Examples:

Machine Translation → English → French

Text Summarization → Long text → Short summary / Key Features

Dialogue Systems → Response generation

(D) Numeric / Scalar Output

- Model outputs a score/number.

Examples:

Similarity score between two sentences (0 to 1)

Sentiment score (e.g., +0.85 positive)

Language model perplexity

(E) Knowledge-based Output

- Output is a fact, relation, or graph entry.

Examples:

Information Extraction → (Obama, bornIn, Hawaii)

Knowledge Graph completion

Tools for Co-domain Handling in NLP

- Different tools/frameworks are used depending on the type of co-domain:

Categorical Co-domains

Scikit-learn → text classification **spaCy / NLTK** → POS tagging, NER

Hugging Face Transformers → classification tasks

Structured Co-domains

Stanford CoreNLP → Dependency parsing, constituency parsing

spaCy → Dependency relations, NER **AllenNLP** → Semantic role labeling

Sequence Co-domains

Hugging Face Transformers (BERT, GPT, T5, MarianMT) → MT, summarization, QA

OpenNMT / Fairseq → Machine Translation

Numeric Co-domains

Sentence-BERT → Similarity scores

Word2Vec, GloVe, FastText → Vector embeddings

Knowledge-based Co-domains

WordNet → Synonymy, hypernymy relations

Neo4j / RDF Tools → Store extracted knowledge

UMLS (medical), Wikidata → Knowledge graph reasoning

Applications of Co-domain in NLP

Classification (Categorical)

Sentiment Analysis, Spam Detection, POS tagging

Parsing / Information Extraction (Structured)

Dependency parsing for grammar checking

NER (extracting entities from contracts, medical reports)

Sequence Generation

Machine Translation (English → German)

Summarization (news → abstract)

Dialogue Systems (chatbots, assistants)

Scoring & Ranking

Search engines (query–document similarity)

Semantic similarity tasks (paraphrase detection)

Knowledge Graphs

Fact extraction (Person–Location–Relation)

Ontology building in specialized domains (law, medicine)

Domain Vs Co-domain

Aspect	Domain (Input Side)	Co-domain (Output Side)
Definition	The set of possible inputs (text/speech data given to NLP system).	The set of possible outputs (labels, sequences, structures, or values produced).
Nature	Usually unstructured raw data (text, audio, queries).	Usually structured results (labels, graphs, scores, generated text).
Examples	- English sentences - Tweets - Medical/legal documents - Customer reviews - Spoken transcripts	- Sentiment labels {Positive, Negative} - Translated text (English → French) - Named entities (PERSON, LOCATION) - Parse trees - Similarity score (0.85)
Types	- General-domain (news, Wikipedia) - Domain-specific (law, medicine) - Task-specific (tweets, transcripts)	- Categorical (classification) - Structured (parsing, NER) - Sequence (translation, summarization) - Numeric (similarity, sentiment score) - Knowledge-based (facts, relations)
Tools	- NLTK (tokenization, corpora) - spaCy (pre-processing, embedding) - Stanford CoreNLP (linguistic features) - OpenNLP (tokenizer, POS)	- Scikit-learn (classification outputs) - Hugging Face Transformers (translation, summarization) - AllenNLP (semantic role labeling) - Neo4j / RDF (knowledge graph outputs) - Sentence-BERT (numeric similarity)
Applications	- Input: Tweets → detect hate speech - Input: Medical records → disease prediction - Input: Court documents → legal research	- Output: Hate/Not Hate (categorical) - Output: Predicted disease (categorical) - Output: Extracted case entities (structured) - Output: Summarized judgment (sequence)

3.7 Binary Relations : Symmetry, Reflexivity, Transitivity

3.7.1 Symmetric Relation

- Bi-directional relations
- Synonymy
- Semantic similarity
- Co-reference (*Alice = She*).

  If (a, b) ∈ R, then (b , a) ∈ R.

- Many relations in semantics are symmetric.

Example:

- ✓ *Synonymy* → If "big" is a synonym of "large," then "large" is a synonym of "big."
- ✓ *Semantic similarity* → If Doc1 ≈ Doc2, then Doc2 ≈ Doc1.

Types of Symmetry in NLP

(A) Lexical Symmetry

Relations between words.

Example: Synonyms ("big" ↔ "large"), antonyms (though sometimes not perfectly symmetric).

(B) Syntactic Symmetry

Relations between sentence structures.

Example: Active voice ↔ Passive voice (meaning preserved).

"The cat chased the mouse." ↔ "The mouse was chased by the cat."

(C) Semantic Symmetry

Sentence meaning equivalence.

Example: Paraphrases →

Text 1: "The boy is eating an apple."

Text 2: "An apple is being eaten by the boy."

(D) Pragmatic Symmetry

Conversational / discourse-level symmetry.

Example: Question–Answer equivalence in dialogue systems.

User: “Can you tell me the time?”

System: “Yes, it’s 5 PM.” ↔ “It’s 5 PM.”

Tools for Symmetry in NLP

- ✓ WordNet → Lexical symmetry (synonym/antonym relations).
- ✓ NLTK → Synonym/antonym checks.
- ✓ spaCy → Semantic similarity scoring (cosine of embeddings).
- ✓ Sentence-BERT (SBERT) → Symmetric similarity of sentence pairs.
- ✓ Hugging Face Transformers → Paraphrase detection (e.g., paraphrase-MiniLM).
- ✓ Translation APIs (MarianMT, Fairseq, Google Translate) → Translation symmetry.
- ✓ Neo4j / RDF triple stores → Representing symmetric relations in knowledge graphs.

Applications of Symmetry in NLP

Synonym detection / Thesaurus building

Identifying bidirectional synonym relations in WordNet.

Paraphrase identification

Checking if two sentences are semantically equivalent.

Semantic similarity & clustering

Grouping similar documents or sentences.

Machine Translation

Ensuring translation pairs are symmetric (EN↔FR dictionary).

Information Retrieval

If doc A is relevant to query B, query B should also retrieve doc A.

Dialogue Systems

Handling paraphrased queries symmetrically ("Where's my order?" ↔ "Track my package").

3.7.2 Reflexive Relation

- Every element relates to itself.
- Self-relations
- Word sense disambiguation (a word always maps to itself as a possible sense).

  (a, a) ∈ R for all 'a'.
- Some linguistic/semantic relations are reflexive.

Example:

- ✓ *Synonymy with itself* → Every word is synonymous with itself.
- ✓ Sentence entailment: *A sentence always entails itself.*

Types in NLP

- Lexical Reflexivity → Every word = itself (*word2vec similarity(word, word) = 1*).
- Syntactic Reflexivity → Reflexive pronouns: *himself, herself*.
- Semantic Reflexivity → Sentences entail themselves ($A \Rightarrow A$).
- Pragmatic Reflexivity → Entities refer to themselves in discourse.

Applications

- Word embeddings (self-similarity = max).
- Sentence similarity & paraphrase detection (baseline check).
- Coreference resolution (entity consistency).
- Document retrieval (doc is always relevant to itself).
- QA/MT (identity translation & entailment).

Tools

- NLTK / spaCy → reflexive pronoun parsing.
- SBERT / Hugging Face → self-similarity embeddings.
- WordNet → lexical identity.
- CoreNLP / AllenNLP → coreference.

3.7.3 Transitive Relation

- Hierarchical relations
- Hyponymy / Hypernymy
- Entailment
- Knowledge graph reasoning.

  If (a, b) ∈ R and (b, c) ∈ R, then (a, c) ∈ R.
- Some relations carry over transitively.

Example:

Hyponymy (is-a hierarchy)

Dog is-an Animal,

Animal is-a Living Being,

⇒ Dog is-a Living Being.

Entailment:

If A ⇒ B, and B ⇒ C, then A ⇒ C.

Types in NLP

- Lexical Transitivity → hypernyms/hyponyms in WordNet
  (dog → mammal → animal).
- Syntactic Transitivity → transitive verbs (requires object: “read a book”).
- Semantic Transitivity → entailment & inference chains.
- Pragmatic Transitivity → coreference chains in discourse.

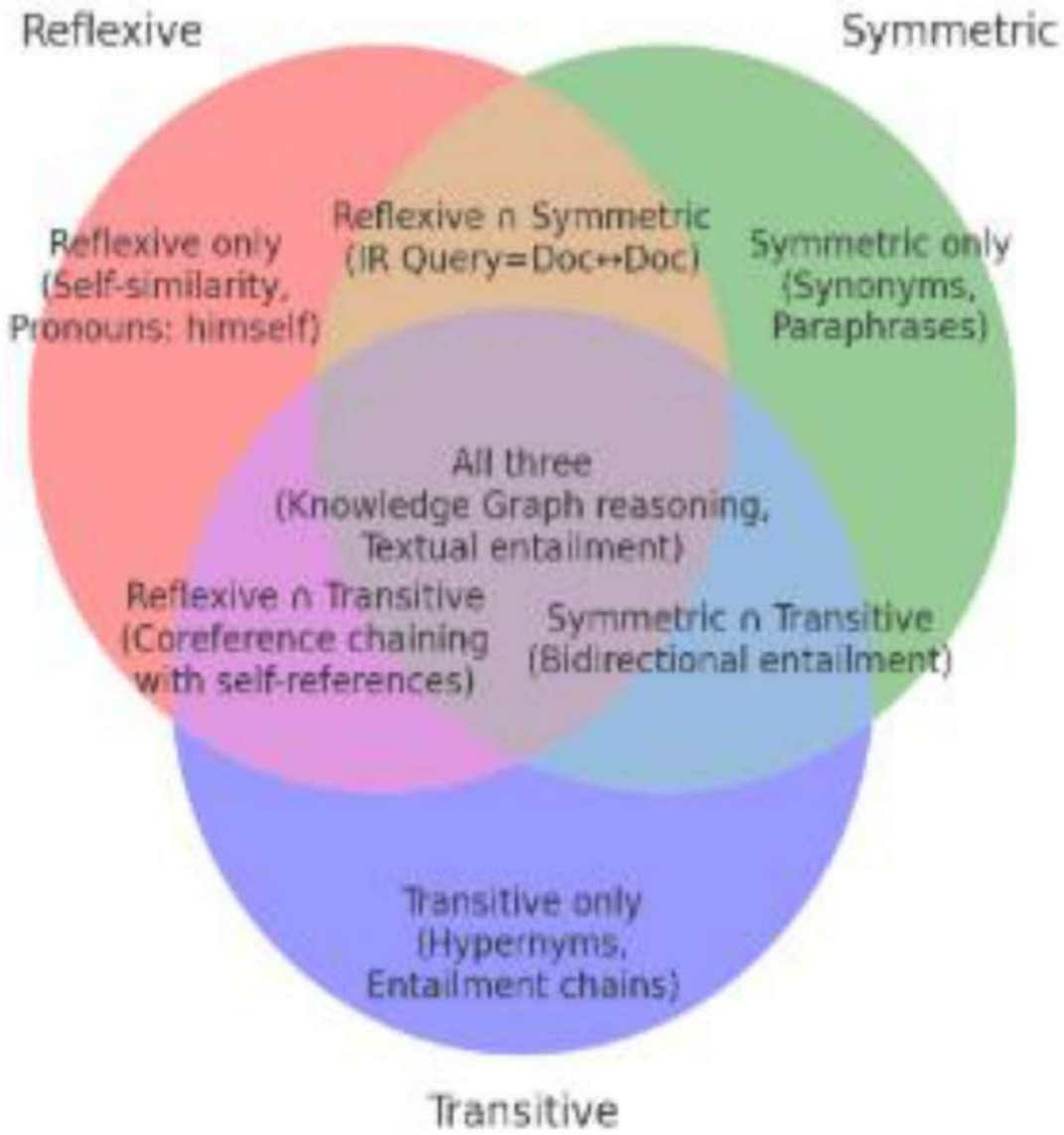
Applications

- Textual entailment (NLI tasks).
- Knowledge graph reasoning (grandparent inference).
- Ontology building (WordNet, taxonomies).
- Coreference resolution (chaining references).
- IR query expansion (dog → mammal → animal).

Tools

- WordNet / ConceptNet → lexical hierarchy.
- AllenNLP / Hugging Face (MNLI) → entailment models.
- Stanford CoreNLP → coreference & dependency parsing.
- Neo4j, RDF, OWL → transitive closure in graphs.
- Prolog → logical transitivity reasoning.

Venn Diagram of Reflexive, Symmetric, and Transitive Relations in NLP



Reflexive Vs Symmetric Vs Transitive

Property	Definition	NLP Examples	Types	Applications	Tools
Reflexive	(a, a) ∈ R for all 'a'	word = word, sentence entails itself, doc = doc	Lexical, Syntactic (pronouns), Semantic, Pragmatic.	Word embeddings, similarity checks, IR baseline, coreference	NLTK, spaCy, SBERT, CoreNLP
Symmetric	(a, b) ∈ R ⇒ (b, a) ∈ R	synonym (happy↔joyful) , paraphrase (A ↔ B)	Lexical, Syntactic, Semantic, Pragmatic.	Synonym detection, paraphrasing, IR query-doc, MT	WordNet, SBERT, Transformers, ConceptNet
Transitive	(a,b)∈R ∧ (b,c)∈R ⇒ (a, c) ∈ R	dog→mammal →animal, John=he, he=teacher ⇒ John=teacher	Lexical, Syntactic (verbs), Semantic, Pragmatic.	Textual entailment, KG reasoning, ontology, coreference	WordNet, AllenNLP, Hugging Face (NLI), Neo4j, Prolog

3.8 Characteristic Function

- A characteristic Function is a binary indicator (1/0) that shows whether a property holds for an element.
- In mathematics (set theory), a characteristic function (also called an indicator function) of a set A is defined as:

$$\chi_A(x) = \begin{cases} 1 & \text{if } x \in A \\ 0 & \text{if } x \notin A \end{cases}$$

- It "characterizes" membership of an element in a set → whether something belongs (1) or does not belong (0).
- In NLP, it underlies binary features, IR relevance, lexical membership, synonym detection, entailment, and coreference.

Use of Characteristic Function in NLP

In Natural Language, we can use characteristic functions to model:

- ✓ Whether a word/token belongs to a vocabulary?
- ✓ Whether a sentence has a particular linguistic property (e.g., passive voice, question form)?
- ✓ Whether a relation holds between two words (e.g., synonymy, entailment)?
- ✓ Whether a document is relevant to a query (IR models)?

Types of Characteristic Functions in NLP

Lexical Characteristic Functions

Check if a word belongs to a dictionary, vocabulary, or keyword list.

Example: χ sports("football")=1.

Syntactic Characteristic Functions

Identify if a sentence has a certain structure.

Example: χ question("Is it raining?")=1.

Semantic Characteristic Functions

Capture meaning-based relations.

Example: χ synonym(happy, joyful)=1.

Pragmatic/Discourse Characteristic Functions

Check whether a discourse relation holds.

Example: χcoreference("John", "he")=1.

Applications in NLP

Information Retrieval (IR):

Relevance indicator: query term $\in$ document = 1.

Boolean retrieval models.

Text Classification:

Bag-of-words representation: each word = 0/1 (present or not).

Sentiment Analysis:

Characteristic function for positive/negative lexicon membership.

Semantic Relations:

Synonymy, hypernymy, entailment relations marked by 0/1 functions.

Coreference Resolution:

Characteristic function for whether two mentions point to the same entity.

Topic Modeling & Word Features:

Binary feature extraction (word present = 1, absent = 0).

Tools Supporting Characteristic Functions in NLP

- NLTK → text preprocessing, lexical sets, Boolean features.
- spaCy → POS tagging, syntactic checks.
- Scikit-learn → binary feature vectors (CountVectorizer).
- WordNet (NLTK interface) → synonym/hypernym indicator functions.
- AllenNLP, Hugging Face Transformers → entailment/semantic relation checks.
- Neo4j / RDF → binary relation indicator in knowledge graphs.

3.9 Function Vs Relation

Aspect	Function in NLP	Relation in NLP
Definition	A mapping where each input has exactly one output. $f : X \rightarrow Y$	A mapping where each input can be related to multiple outputs or none. $R \subseteq X \times Y$.
Nature	Deterministic (one-to-one or many-to-one).	Non-deterministic (one-to-many, many-to-many possible).
Types	- Lexical Functions (POS tagging: word → tag) - Semantic Functions (sentence → sentiment score) - Syntactic Functions (parse tree mapping) - Pragmatic Functions (utterance → speech act)	- Lexical Relations (synonymy, antonymy, hyponymy) - Syntactic Relations (subject-verb-object) - Semantic Relations (entailment, similarity) - Discourse Relations (coreference, coherence)
Examples	- Token → Lemma (running → run) - Word → POS (play → Verb) - Sentence → Sentiment (positive) - English text → French translation	- Synonym(happy, joyful) - Hypernym(dog, animal) - Entailment("Socrates is human" → "Socrates is mortal") - Coreference("John" ↔ "he")

Contd..

Aspect	Function in NLP	Relation in NLP
Applications	- Machine Translation (input → output) - POS tagging - Lemmatization - Sentiment Analysis - Speech-to-text, text-to-speech	- WordNet-based semantic search - Question Answering (entity relations) - Knowledge Graph construction - Coreference resolution - Textual Entailment & Similarity
Tools	- NLTK, spaCy (lemmatization, POS tagging) - Scikit-learn (classification as functions) - Hugging Face Transformers (translation, summarization) - AllenNLP (semantic role functions)	- WordNet, ConceptNet (lexical/semantic relations) - Neo4j, RDF/OWL (knowledge graphs) - Stanford CoreNLP (coreference relations) - Sentence-BERT (similarity relations) - AllenNLP / NLI models (entailment relations)

Thank You !